

ESPRIT — Stage de fin d'études

Rapport technique détaillé

POC Vérification IA des dossiers d'admission

Travail réalisé et travail restant — 3 septembre 2026 (met à jour la version du 11 août 2026)

Dépôt : github.com/ab2sfaxii/poc-admission-esprit (branche main)

Ce rapport détaille, module par module, l'ensemble du travail réalisé depuis le début du stage (Sprint 1, Sprint 2 et Sprint 3), les choix techniques et les difficultés concrètement rencontrées et résolues, ainsi que tout ce qui reste à faire pour compléter le projet.

Sommaire

1. Contexte et objectifs du POC
2. Sprint 1 — Prétraitement OpenCV et OCR Tesseract
3. Sprint 2 — Classification et extraction structurée
4. Sprint 3 — Cohérence, complétude, entretien et quiz candidat
5. Interface Streamlit (espace enseignant / espace candidat)
6. Difficultés techniques rencontrées et résolues
7. Stratégie de tests et qualité
8. Gestion de version (Git)
9. Ce qu'il reste à faire
10. Décisions et données encore nécessaires
11. Conclusion

1. Contexte et objectifs du POC

Le projet consiste à développer un Proof of Concept (POC) assistant les enseignants d'ESPRIT dans la vérification des dossiers de candidature pour l'admission parallèle en 4ème année du cycle ingénieur. Le système lit automatiquement les documents fournis par un candidat (carte d'identité, acte de naissance, diplôme du baccalauréat, diplôme de licence, relevés de notes), en extrait les informations, vérifie leur cohérence, calcule un score de complétude, et prépare des questions d'entretien adaptées au profil du candidat.

Principe fondamental (rapport de stage, §1.5) : le système n'émet jamais de décision d'admission. Il produit uniquement un rapport structuré qui assiste les enseignants ; la décision reste entièrement humaine.

Le projet suit une démarche agile en trois sprints : Sprint 1 (prétraitement et OCR), Sprint 2 (classification et extraction), Sprint 3 (vérification de cohérence, score de complétude, génération de questions d'entretien). **Ce rapport couvre les Sprints 1 et 2 entièrement terminés, et le Sprint 3 achevé pour tout ce qui ne dépend pas d'une décision métier de l'encadrante** (barème officiel, niveau académique conforme, nationalités prioritaires — cf. §9 et §10).

2. Sprint 1 — Prétraitement OpenCV et OCR Tesseract

2.1 Modules livrés

Fichier	Rôle
preprocessing/image_utils.py	Chargement image, niveaux de gris, contraste adaptatif (CLAHE), débruitage, recadrage, redimensionnement, binarisation (disponible mais non appliquée par défaut, cf. §6).
preprocessing/deskew.py	Estimation et correction de l'angle d'inclinaison.
ocr/tesseract_engine.py	Interface Tesseract : texte brut, texte avec confiance par mot, regroupement par ligne positionnelle, seconde passe OCR ciblée (zoom).
ocr/text_normalization.py	Normalisation du texte (minuscules, sans accents) pour la classification et l'extraction.
config/settings.py	Paramètres centralisés (langues OCR, chemin Tesseract, seuils, barème du score de complétude).

2.2 Résultat

Pipeline validé sur 5 documents synthétiques (tous les types) et sur de vrais documents tunisiens (CIN et acte de naissance réels). Langues OCR actives : français, arabe, anglais.

3. Sprint 2 — Classification et extraction structurée

3.1 Classification par règles

classification/rules_classifier.py : classification par mots-clés pondérés pour les 5 types de documents (CIN, acte de naissance, bac, licence, relevé de notes), avec mots-clés arabes dédiés pour la carte d'identité tunisienne.

3.2 Contrat JSON commun

```
{
  "document_type": "cin",
  "fields": { "nom": "...", "prenom": "...", ... },
  "missing_fields": [],
  "warnings": [],
  "extraction_status": "success"
}
```

3.3 Les 5 extracteurs spécialisés

Type	Fichier	Champs requis
cin	extraction/cin_extractor.py	nom, prenom, date_naissance, numero_document
acte_naissance	extraction/acte_naissance_extractor.py	nom, prenom, date_naissance
bac	extraction/bac_extractor.py	nom, prenom, annee_obtention
diplome_licence	extraction/licence_extractor.py	nom, prenom, specialite, annee_obtention
releve_notes	extraction/releve_extractor.py	annee_universitaire, moyenne

Tous les extracteurs héritent de `extraction/base_extractor.py` (`BaseExtractor`), garantissant un contrat de sortie identique quel que soit le type de document.

4. Sprint 3 — Cohérence, complétude, entretien et quiz candidat

Le Sprint 3 comprend 4 vérifications automatiques (identité, années universitaires, doublons, complétude), un score chiffré agrégeant les trois premières, et le module de génération de questions d'entretien. Chaque vérification est validée à deux niveaux : 60 tests unitaires (cas construits à la main) et une évaluation à grande échelle sur un jeu de 1000 dossiers synthétiques avec vérité terrain (cf. §7).

4.1 Vérification de cohérence d'identité

`verification/identity_checker.py` compare le nom et le prénom entre tous les documents du dossier qui exposent un champ d'identité exploitable (directement, ou via le champ "etudiant" d'un relevé de notes). La comparaison utilise la similarité de l'ensemble des mots normalisés (indice de Jaccard) plutôt qu'une égalité stricte, pour tolérer les petites variations d'OCR, et détecte séparément une inversion nom/prénom (mêmes mots, ordre différent) par comparaison de séquence.

Nouveau : le module compare également la **date de naissance** entre la CIN et l'acte de naissance (égalité stricte après normalisation du séparateur — une date n'a pas de variante légitime, contrairement à un nom). Ajout à faible risque : contrairement au score de complétude ou au niveau académique, aucun barème n'était nécessaire.

Le système signale toute incohérence sans jamais rejeter automatiquement un dossier (principe de non-substitution). La génération de questions d'entretien est en revanche bloquée tant que l'incohérence n'est pas résolue.

Limite connue : la comparaison de noms est fiable au sein d'une même écriture, mais ne peut pas rapprocher un nom écrit en arabe de sa translittération française (nécessiterait une translittération dédiée, non implémentée).

4.2 Score de complétude global du dossier (nouveau)

`verification/completeness_score.py` combine trois signaux déjà disponibles du pipeline en une note unique (0-100%) et un statut :

- **Présence des documents** (poids 50%) — proportion des emplacements du dossier effectivement renseignés.
- **Complétude de l'extraction** (poids 30%) — proportion des champs attendus réellement extraits sur les documents fournis.
- **Cohérence** (poids 20%) — proportion des 3 vérifications (identité, années, doublons) qui ne signalent aucune anomalie.

Statut dérivé : `documents_manquants > anomalie_detectee > extraction_incomplete > complet` (le premier applicable l'emporte). Le barème (pondération ci-dessus) est une **valeur par défaut du POC, documentée et ajustable** dans `config/settings.py` — aucun barème officiel n'ayant été fourni par l'encadrante à ce jour (cf. §10). Le niveau académique (moyennes minimales, mentions attendues) reste volontairement hors périmètre de ce score.

4.3 Vérification des années universitaires (nouveau)

`verification/academic_years_checker.py` compare les années universitaires des relevés d'un dossier entre elles, et avec l'année d'obtention de la licence : détecte un doublon d'année universitaire, un relevé manquant dans le parcours L1→L2→L3→M1, une discontinuité entre deux niveaux adjacents, et une incohérence entre la fin de la L3 et l'année d'obtention de la licence.

4.4 Détection de documents dupliqués (nouveau)

verification/duplicate_document_checker.py compare le texte OCR brut de chaque paire de documents du dossier (indépendamment du type de document déjà reconnu), pour détecter un même document déposé deux fois par erreur dans des emplacements différents.

4.5 Génération de questions d'entretien (Module 2)

interview/question_generator.py construit un profil candidat structuré (spécialité, établissement, résultats académiques, matières les mieux évaluées) à partir des champs déjà extraits, puis interroge un LLM pour générer des questions d'entretien personnalisées. Le module ne renvoie jamais de questions génériques inventées localement en l'absence de clé API : il échoue explicitement, avec un message clair.

4.6 Trois fournisseurs LLM pris en charge

Fournisseur	Modèle par défaut	Remarque
Groq (recommandé)	llama-3.3-70b-versatile	Gratuit, sans carte bancaire, 30 req/min - aucun problème de quota par modèle observé.
Google Gemini	gemini-2.0-flash (plusieurs au choix)	Palier gratuit sans carte, mais quota "limit: 0" possible selon le compte/projet Google Cloud - sélecteur de modèle ajouté pour contourner.
OpenAI	gpt-4o-mini	Nécessite un compte facturé (carte bancaire, crédit prépayé).

Mise à jour : au 11 août, ce module était bloqué par un compte OpenAI sans crédit ("insufficient_quota"). Groq et Gemini ont depuis été ajoutés comme fournisseurs gratuits, débloquent la génération réelle de questions sans dépendre d'un budget API.

4.7 Espace Candidat et quiz de pré-entretien

Fonctionnalité ajoutée à la demande de l'encadrante : un espace séparé (pages/1_Espace_Candidat.py, application Streamlit multipage) où le candidat s'entraîne avant son entretien réel.

- Le candidat s'identifie par son nom complet.
- Si son dossier a déjà été vérifié côté enseignant, son profil est automatiquement retrouvé (aucun re-dépôt de documents nécessaire).
- Sinon, il peut déposer lui-même son bac/licence/relevé pour construire son profil.
- Un quiz à choix multiples (QCM), généré par LLM à partir de ce profil, est ensuite proposé - correction automatique, aucun appel API supplémentaire nécessaire pour noter les réponses.
- Le score est enregistré et redevient visible dans l'espace enseignant, en regard du dossier du candidat.

interview/quiz_store.py assure la persistance (fichiers JSON locaux, jamais versionnés car contenant des données personnelles) reliant les deux espaces via une clé dérivée du nom du candidat.

5. Interface Streamlit

5.1 Espace enseignant (app.py)

- Un emplacement de dépôt dédié par type de document (au lieu d'un téléversement générique), à l'image d'un vrai dossier de candidature.
- Détection automatique des documents déposés dans le mauvais emplacement (type détecté $\neq$ type attendu).
- **Score de complétude chiffré et statut du dossier (nouveau)**, avec la liste des documents manquants.
- Vérification de cohérence d'identité (nom et date de naissance), des années universitaires, et détection de doublons.
- Score du quiz de pré-entretien du candidat, une fois complété.
- Génération des questions d'entretien (30 par défaut, configurable).

5.2 Espace candidat (pages/1_Espace_Candidat.py)

Décrit en détail au §4.7. Séparé de l'espace enseignant via la navigation multipage native de Streamlit.

5.3 Identité visuelle

ui_common.py factorise le thème visuel (couleurs et logo ESPRIT), les emplacements de documents, l'exécution mise en cache du pipeline et le sélecteur de fournisseur LLM, réutilisés par les deux espaces.

6. Difficultés techniques rencontrées et résolues

Contraste sur documents réels

L'égalisation d'histogramme globale (cv2.equalizeHist) détruisait le texte sur de vraies photos à éclairage inégal. Remplacée par CLAHE (contraste adaptatif local).

Binarisation contre-productive

La binarisation globale (Otsu), appliquée par défaut, faisait disparaître le texte sur de vraies photos. Retirée du pipeline par défaut : Tesseract lit mieux un niveau de gris bien contrasté.

Bug de correction d'inclinaison

Détecté par les tests : la rotation de correction était appliquée dans le même sens que le biais mesuré, doublant l'inclinaison au lieu de la corriger. Signe inversé.

Documents en écriture arabe (CIN tunisienne)

Le format "Libellé : valeur" ne s'applique pas : le libellé est imprimé à droite de la valeur sur la même ligne (RTL). Extraction positionnelle développée (mots regroupés par ligne, triés par position), avec repli sur la position relative des champs et une seconde passe OCR ciblée (zoom) sur les zones trop dégradées.

Tableaux français (acte de naissance)

Format tabulaire où le libellé et sa valeur sont parfois séparés sur deux lignes OCR distinctes.
Extraction positionnelle symétrique développée.

Documents personnels et confidentialité

Le dépôt étant public, les vrais documents de test (CIN, actes de naissance) sont explicitement exclus du suivi Git (.gitignore) et ne quittent jamais le poste local.

Interception HTTPS locale (SSL)

Les appels aux API LLM échouaient avec CERTIFICATE_VERIFY_FAILED sur ce poste, un antivirus/proxy local interceptant le trafic HTTPS. Résolu avec la bibliothèque truststore (vérification déléguée au magasin de certificats natif de l'OS).

Quota gratuit "limit: 0" sur Gemini

Certains comptes Google reçoivent un quota gratuit nul pour un ou plusieurs modèles Gemini. Un sélecteur de modèle a été ajouté, puis un troisième fournisseur (Groq) proposé par défaut, qui ne présente pas cette limitation.

Barème absent pour la complétude et les mentions (nouveau)

Une analyse statistique du jeu de 1000 dossiers a montré que la moyenne des relevés se chevauche entre mentions voisines du diplôme (ex. "Assez Bien" 10.47-13.40 vs "Bien" 12.46-15.91 sur les dossiers cohérents) : un seuil fixe créerait de faux positifs. Décision : ne pas coder de règle sans barème officiel de l'encadrante, plutôt que d'inventer un seuil non fiable (cf. §9, anomalie moyenne_incoherente_avec_mention).

7. Stratégie de tests et qualité

60 tests automatisés (pytest, contre 38 au 11 août), tous passants : prétraitement, OCR, classification, extraction, pipeline de bout en bout, et les 4 vérifications de cohérence/complétude du Sprint 3. Les documents synthétiques sont générés par tests/sample_documents.py, faute de vrais documents fournis pour l'ensemble des types (cf. §10).

La suite de tests a permis de détecter et corriger le bug de correction d'inclinaison (§6) avant qu'il n'affecte la démonstration.

7.1 Validation à grande échelle (nouveau depuis le 11 août)

tests/evaluate_pipeline.py fait tourner le pipeline réel sur un échantillon du jeu de 1000 dossiers synthétiques avec vérité terrain (data/samples_dataset/, 7 catégories : 1 "coherent" et 6 avec une anomalie ciblée) et mesure, par catégorie, si chaque module détecte l'anomalie qui lui correspond sans se déclencher ailleurs. Résultat sur un échantillon de 105 dossiers (15 par catégorie) :

Module	Anomalie ciblée (attendu : détection basse)	Résultat
completeness_score	document_bac_manquant	100% détecté, 0% de faux positifs ailleurs
academic_years_checker	annee_universitaire_duplquee, annee_licence_incoherente	100% détecté sur les 2 catégories, ~100% ailleurs
identity_checker	nom_mal_orthographie, identite_releve_inversee	100% détecté (l'inversion nom/prénom, invisible à une comparaison par ensemble, n'était détectée qu'à 13% avant l'ajout de la comparaison de séquence)
duplicate_document_checker	(aucun scénario dédié dans ce dataset)	0% de faux positifs sur les 7 catégories

Ce jeu de 1000 dossiers comble un manque identifié dans la version précédente du rapport (§7) : les tests unitaires seuls ne portent que sur des cas construits à la main et ne mesurent pas la fiabilité réelle du pipeline à l'échelle.

8. Gestion de version (Git)

<https://github.com/ab2sfaxii/poc-admission-esprit>
Branche : main

Dépôt dédié, indépendant du dépôt d'équipe PIWeb. Historique de commits complet retraçant chaque étape (Sprint 1 & 2, corrections OCR, vérifications de cohérence, score de complétude, module LLM, interface, corrections SSL...). Aucune donnée personnelle (vrais documents, profils candidats) n'est versionnée.

9. Ce qu'il reste à faire

Fonctionnalité	État
Score de complétude global du dossier	Implémenté - barème par défaut du POC, non validé par l'encadrante

Fonctionnalité	État
Vérification des années universitaires	Implémenté
Détection de documents dupliqués	Implémenté
Cohérence d'identité (nom + date de naissance)	Implémenté
Génération réelle de questions d'entretien	Fonctionnel - Groq/Gemini gratuits, plus besoin de crédit OpenAI
Vérification du niveau académique conforme	Non implémenté - critères non définis
Anomalie "moyenne incohérente avec la mention"	Étudiée, non implémentée - seuil fiable impossible à dériver sans barème officiel (cf. §6)
Classifieur par Machine Learning (évolution proposée)	Non implémenté - classification 100% par règles actuellement, hors périmètre POC
Jeu de données de test réel et diversifié	Toujours seulement 2 vrais documents testés (CIN, acte de naissance)

10. Décisions et données encore nécessaires

Inchangé depuis le 31 juillet : ces points bloquent toujours la suite et ne peuvent pas être devinés sans risquer un développement à refaire.

- Documents réels anonymisés (ou caviardés) par type, en quantité et qualité variées.
- Dossiers candidats complets, valides ET volontairement invalides (pour tester la détection d'anomalies sur données réelles, pas seulement synthétiques).
- Validation ou correction du barème par défaut utilisé pour le score de complétude (poids par type d'anomalie, seuils de statut) - config/settings.py.
- Définition précise du niveau académique conforme et des documents obligatoires.
- Nationalités prioritaires à couvrir (impact majeur sur la charge de travail, comme vécu avec la CIN en arabe).
- Confirmation du cadre légal d'utilisation de vrais documents de candidats, même anonymisés.

11. Conclusion

Les Sprints 1 et 2 sont entièrement terminés et testés. Le Sprint 3 est désormais achevé pour tout ce qui ne dépend pas d'une décision métier externe : vérification de cohérence d'identité (nom et date de naissance), des années universitaires et des doublons, score de complétude chiffré du dossier, génération de questions d'entretien via LLM (trois fournisseurs, dont deux gratuits), et l'espace candidat avec quiz de pré-entretien auto-corrigé.

Chaque vérification est validée à deux niveaux : 60 tests unitaires, et une évaluation à grande échelle sur un jeu de 1000 dossiers synthétiques avec vérité terrain, montrant une détection proche de 100% sur l'anomalie ciblée et 0% de faux positifs ailleurs pour chaque module (§7.1).

Le travail restant ne porte plus sur du code, mais sur des **décisions métier que seule l'encadrante peut fournir** : barème officiel du score de complétude, définition du niveau académique conforme, nationalités prioritaires, et accès à des documents réels pour valider le pipeline au-delà des données synthétiques - détaillées au chapitre 10.

Rapport généré à partir de l'état réel du dépôt au 3 septembre 2026.